

CORRECTION – Examen du Premier Semestre

UE : Systèmes Logiques Programmables — Mars 2026

Exercice 1 (5 pts) – Généralités sur les PLD, FSM et VHDL

Question 1 – Définir PLD et présenter deux circuits intégrés de la famille des PLD

PLD (Programmable Logic Device) : circuit intégré dont la fonction logique n'est pas fixée à la fabrication mais peut être configurée/programmée par l'utilisateur après fabrication, à l'aide d'outils logiciels de synthèse.

Deux exemples de circuits PLD :

- **PAL (Programmable Array Logic)** : réseau ET programmable + réseau OU fixe. L'utilisateur programme les connexions du plan ET pour réaliser ses équations logiques. Exemple : PAL16L8.
- **FPGA (Field Programmable Gate Array)** : composant contenant des milliers de blocs logiques configurables (CLB/LUT), des bascules, des blocs mémoire et des ressources d'interconnexion programmables. Exemple : Xilinx Spartan-7.

Question 2 – Différence entre circuit intégré à fonctionnement programmable et circuit intégré à architecture programmable

- **Circuit à fonctionnement programmable** (ex. microprocesseur, microcontrôleur) : l'architecture matérielle (ALU, registres, mémoire) est figée. C'est le *programme logiciel* (instructions) chargé en mémoire qui détermine le comportement. La flexibilité est obtenue par le logiciel.
- **Circuit à architecture programmable** (ex. FPGA, PLD) : c'est la *structure matérielle elle-même* (portes, interconnexions) qui est reconfigurée. Il n'y a pas d'exécution séquentielle d'instructions : le traitement est entièrement parallèle et câblé.

Question 3 – Qu'est-ce qui caractérise un état dans une machine à États finis ?

Un **état** dans une machine à états finis (FSM) représente une situation stable du système à un instant donné. Il est caractérisé par :

- Les **valeurs des sorties** associées (dans une machine de Moore, les sorties ne dépendent que de l'état ; dans une machine de Mealy, elles dépendent aussi des entrées).
- L'ensemble des **transitions** possibles vers d'autres états, conditionnées par les valeurs des entrées.
- Un **codage binaire** (vecteur de bits) permettant de mémoriser l'état courant dans des bascules.

Question 4 – Niveaux de description VHDL pour décrire un circuit logique

VHDL offre principalement **trois niveaux** de description :

1. **Niveau comportemental (Behavioral)** : on décrit le comportement algorithmique du circuit (processus, instructions séquentielles) sans se soucier de l'implémentation matérielle. C'est le niveau le plus abstrait.
2. **Niveau flot de données (Data Flow)** : on décrit les transferts de données entre registres via des équations logiques concurrentes (affectations concurrentes de signaux).
3. **Niveau structurel (Structural)** : on décrit le circuit comme une interconnexion de composants (instanciation de portes, bascules, sous-modules). C'est le plus proche du schéma électronique.

Question 5 – Caractéristiques d'une mémoire : temps d'accès, temps de cycle, débit

- **Temps d'accès (t_{acc})** : temps écoulé entre l'instant où l'adresse est présentée à la mémoire et l'instant où la donnée est disponible en sortie. Il caractérise la *latence* de la mémoire.

- **Temps de cycle (t_{cycle})** : durée minimale entre deux accès consécutifs à la mémoire. Il est toujours $\geq t_{\text{acc}}$ car certaines mémoires nécessitent un temps de recharge/restauration après chaque accès.
- **Débit (D)** : quantité d'information transférée par unité de temps. $D = \text{taille du mot} / t_{\text{cycle}}$. Il exprime la *bande passante* effective de la mémoire.

Exercice 2 (4 pts) – Machine distributrice de Pop-Corn en VHDL

Analyse du problème

La machine accepte des pièces de 50 FCFA et 100 FCFA. Elle délivre une boîte de Pop-Corn dès que la somme accumulée atteint 150 FCFA, et rend la monnaie si nécessaire (cas 100+100 = 200 FCFA → rendre 50 FCFA).

Entrées : P50 (pièce 50 FCFA), P100 (pièce 100 FCFA) **Sorties** : CORN (délivrer Pop-Corn), R50 (rendre 50 FCFA)

Question 1 – Graphe d'état de Moore du contrôleur

On définit les états selon la somme cumulée :

État	Somme cumulée	Sorties (CORN / R50)	Description
S0	0 FCFA	0 / 0	État initial (attente)
S1	50 FCFA	0 / 0	Une pièce de 50 FCFA insérée
S2	100 FCFA	0 / 0	100 FCFA accumulés
S3	150 FCFA	1 / 0	Délivrer Pop-Corn, pas de monnaie
S4	200 FCFA	1 / 1	Délivrer Pop-Corn + rendre 50 FCFA

Transitions :

S0 → [P50] → S1 | S0 → [P100] → S2

S1 → [P50] → S2 | S1 → [P100] → S3

S2 → [P50] → S3 | S2 → [P100] → S4

S3 → S0 (après un cycle d'horloge : CORN délivré, retour à l'état initial)

S4 → S0 (après un cycle d'horloge : CORN délivré + R50 rendu)

Question 2 – Implémentation VHDL du contrôleur

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity popcorn_machine is
    Port (
        clk : in  STD_LOGIC;
        rst : in  STD_LOGIC;
        P50 : in  STD_LOGIC;  -- piece 50 FCFA
        P100 : in  STD_LOGIC; -- piece 100 FCFA
        CORN : out STD_LOGIC;  -- delivrer Pop-Corn
        R50 : out STD_LOGIC;   -- rendre 50 FCFA
    );
end popcorn_machine;

architecture Behavioral of popcorn_machine is
    type state_type is (S0, S1, S2, S3, S4);
    signal current_state, next_state : state_type;

```

```

begin
  -- Registre d'etat (sequentiel)
  process(clk, rst)
  begin
    if rst = '1' then
      current_state <= S0;
    elsif rising_edge(clk) then
      current_state <= next_state;
    end if;
  end process;

  -- Logique de transition (combinatoire)
  process(current_state, P50, P100)
  begin
    next_state <= current_state; -- default
    case current_state is
      when S0 =>
        if P50 = '1' then next_state <= S1;
        elsif P100 = '1' then next_state <= S2;
        end if;
      when S1 =>
        if P50 = '1' then next_state <= S2;
        elsif P100 = '1' then next_state <= S3;
        end if;
      when S2 =>
        if P50 = '1' then next_state <= S3;
        elsif P100 = '1' then next_state <= S4;
        end if;
      when S3 => next_state <= S0; -- CORN delivre
      when S4 => next_state <= S0; -- CORN + monnaie
    end case;
  end process;

  -- Logique de sortie de Moore
  CORN <= '1' when (current_state = S3 or current_state = S4) else '0';
  R50 <= '1' when (current_state = S4) else '0';
end Behavioral;

```

Exercice 3 (3 pts) – Module TIMER, $f_{clk} = 100 \text{ MHz}$

Données

Fréquence d'horloge : $f_{clk} = 100 \text{ MHz}$ ■ Période d'horloge : $T_{clk} = 1 / 100 \times 10^6 = 10 \text{ ns}$

Durée à décompter : $D = 30 \text{ ms}$

Question 1 – Nombre m d'impulsions pour décompter D = 30 ms

Le nombre d'impulsions nécessaires est :

$$m = D / T_{clk} = 30 \times 10^{-3} / 10 \times 10^{-9} = 3\,000\,000 \text{ impulsions}$$

$$m = 3 \times 10^6 = 3\,000\,000$$

Question 2 – Taille du bus pour le registre du timer

Il faut un registre capable de compter jusqu'à $m = 3\,000\,000$. On cherche n tel que $2^n \geq 3\,000\,000$:

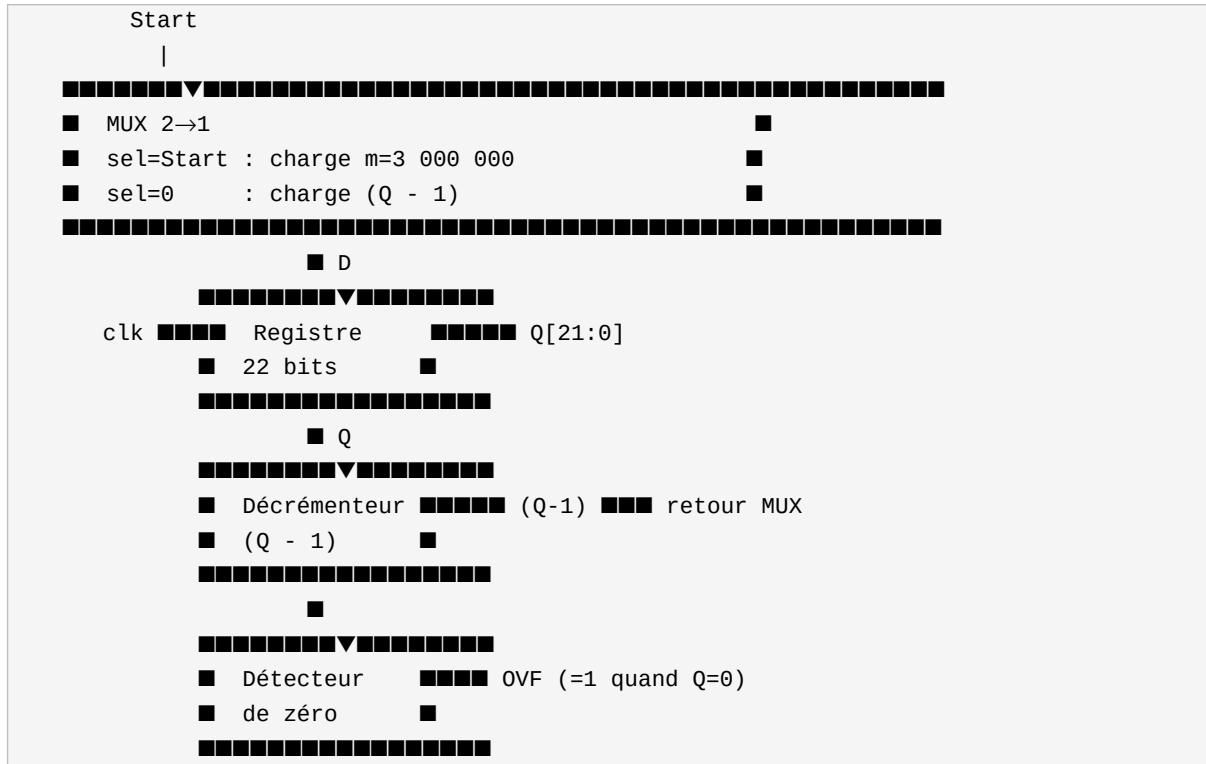
$$2^{21} = 2\,097\,152 < 3\,000\,000 < 4\,194\,304 = 2^{22}$$

■ **n = 22 bits** (un bus de 22 bits est nécessaire pour le registre du timer).

Question 3a – Schéma bloc RTL du TIMER avec décrémenteur

Le TIMER utilise un registre de 22 bits préchargé à la valeur $m = 3\,000\,000$ et décrémente à chaque coup d'horloge. Quand le registre atteint 0, la sortie OVF passe à 1.

Schéma RTL (décrémenteur) :



Question 3b – Schéma bloc RTL du TIMER avec incrémenteur

Le registre est initialisé à 0 et incrémenté à chaque coup d'horloge. Quand la valeur atteint $m = 3\,000\,000$, la sortie OVF passe à 1 et le registre est remis à 0.

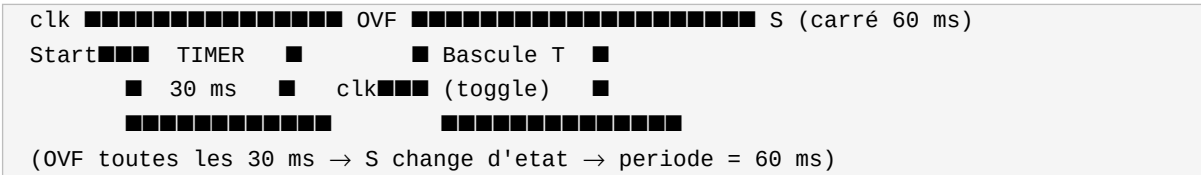


[illegible]

Question 4 – Module RTL générant un signal carré de période 60 ms

Un signal carré de **période 60 ms** a une demi-période de 30 ms. On utilise le TIMER précédent (overflow toutes les 30 ms) et une bascule D qui bascule à chaque OVF :

- À chaque front montant de OVF (toutes les 30 ms), la sortie S est complémentée : $S \leftarrow \text{NOT}(S)$
- Le signal S est un signal carré de période $T = 2 \times 30 \text{ ms} = 60 \text{ ms}$, avec rapport cyclique 50%.



Exercice 4 (8 pts) – Circuit d'émission série asynchrone

Rappel du principe de la transmission série asynchrone

Chaque caractère est encadré par : 1 bit de start (niveau 0), 8 bits de données (D0 à D7, LSB en premier), 1 bit de stop (niveau 1). La ligne est au repos à 1 (état Mark). La trame complète fait 10 bits.

Analyse des signaux de contrôle du registre à décalage

D'après le schéma de la figure 1, le registre à décalage 8 bits est piloté par :

- **SI** : entrée série (donnée à injecter lors du décalage)
- **SO** : sortie série (bit de sortie du registre, connecté à TxD via logique)
- **MC (Mode Control)** : MC=0 → décalage à droite ; MC=1 → chargement parallèle des données D7..D0
- **CK** : horloge de décalage
- **TxClock** : horloge de transmission (issue du diviseur par 16 appliqué à l'horloge système)

Fonctionnement global du circuit d'émission

Phase 1 – Chargement (MC=1) : quand Send est activé, le signal Mode passe à 1. Le registre charge en parallèle les 8 bits de données D7..D0 sur un front d'horloge.

Phase 2 – Émission (MC=0) : Mode revient à 0, le registre décale les bits vers la droite au rythme de TxClock. Les bits sortent par SO : d'abord le bit de start (0 forcé), puis D0..D7 (LSB en premier), puis le bit de stop (1).

Le détecteur de zéro surveille les bits restants dans le registre. Quand tous les bits ont été émis (registre = 0), il signale la fin de transmission et permet de revenir à l'état de repos (TxD = 1).

Rôle du diviseur par 16

Le diviseur de fréquence par 16 génère l'horloge de décalage TxClock à partir de l'horloge système. Par exemple, si $f_{\text{sys}} = 1.8432 \text{ MHz}$, alors $f_{\text{TxClock}} = 115\,200 \text{ Hz}$, ce qui correspond à un débit de 115 200 bauds (standard RS-232). Ce diviseur permet de régler le débit de transmission.

Séquence d'émission d'un caractère – Chronogramme explicatif

Instant	Signal TxD	Événement
t0	1 (repos)	Ligne inactive, attente
t1	0	Bit de START émis (début de trame)
t2..t9	D0→D7	8 bits de données émis, LSB (D0) en premier

t10	1	Bit de STOP émis, fin de trame
t11+	1 (repos)	Retour à l'état de repos

Signaux de contrôle et conditions d'activation

- **Reset** : initialise le circuit, remet le registre à un état connu, TxD = 1.
- **Start** : déclenche une nouvelle émission (chargement des données puis début du décalage).
- **Zéro** : signal interne du détecteur de zéro, indique que le registre est vide (émission terminée).
- **Shift** : autorise le décalage du registre au rythme de TxClock.
- **Mode/Data/Send** : signaux de chargement — Send lance le cycle, Data contient les données, Mode contrôle le mode du registre (chargement vs décalage).

Note sur les fronts d'horloge

D'après l'énoncé, tous les circuits sont actifs sur front descendant (falling edge). Il faut donc câbler les horloges sur les entrées CK actives au front négatif. Cette contrainte est à respecter impérativement lors de l'implémentation RTL ou VHDL.